

Herald — OPERATOR_CREDENTIALS

Herald — Operator Credentials Guide

- Table of contents
- Resolution order (12-factor)
- Setting credentials in ~/.bashrc / ~/.zshrc (shell-export path)
- Setting credentials in .env (project-local path)
- Constitutional rules — what NEVER goes in git
- Per-service credential setup
 - Postgres (LIVE)
 - Redis (LIVE)
 - Telegram — HRD-011 (LIVE)
 - Claude Code dispatcher — HRD-012 (LIVE)
 - Slack — planned, V2
 - Email (SMTP) — planned, V2
 - Email (Resend) — planned, V2
 - Max — planned, V2
 - Microsoft Teams — planned, V3
 - Lark / Discord / WhatsApp / Viber — planned, later iterations
 - Alternate LLM dispatchers (OpenCode / Aider / Gemini / Cursor / Managed Agents) — planned
- Quickstart compose vs native pherald
- Audit checklist (run before every commit)
- Troubleshooting

Herald — Operator Credentials Guide

Field	Value
Revision	1
Created	2026-05-21
Last modified	2026-05-21
Status	active
Status summary	Comprehensive step-by-step guide for obtaining and configuring every environment variable Herald requires — covers live integrations (Postgres, Redis, Telegram, Claude Code) AND reserved env-var names for planned integrations (Slack, Email, Max,

Field	Value
	Microsoft Teams, Lark, Discord, WhatsApp, Viber, OpenCode, Aider, Gemini, Cursor). Documents the dual-source resolution model (shell exports vs <code>.env</code>) per spec §3.3 + Universal Constitution §11.4.10.
Issues	none
Issues summary	—
Fixed	(n/a — new guide)
Continuation	bump when new channel HRDs land (HRD-NNN Slack live → add detailed Slack section; HRD-NNN Email live → SMTP/Resend sections; etc.)

Table of contents

- [Resolution order \(12-factor\)](#)
- [Setting credentials in `~/ .bashrc` / `~/ .zshrc` \(shell-export path\)](#)
- [Setting credentials in `.env` \(project-local path\)](#)
- [Constitutional rules — what NEVER goes in git](#)
- [Per-service credential setup](#)
 - [Postgres \(LIVE\)](#)
 - [Redis \(LIVE\)](#)
 - [Telegram — HRD-011 \(LIVE\)](#)
 - [Claude Code dispatcher — HRD-012 \(LIVE\)](#)
 - [Slack — planned, V2](#)
 - [Email \(SMTP\) — planned, V2](#)
 - [Email \(Resend\) — planned, V2](#)
 - [Max — planned, V2](#)
 - [Microsoft Teams — planned, V3](#)
 - [Lark / Discord / WhatsApp / Viber — planned, later iterations](#)
 - [Alternate LLM dispatchers \(OpenCode / Aider / Gemini / Cursor / Managed Agents\) — planned](#)
- [Quickstart compose vs native pherald](#)
- [Audit checklist \(run before every commit\)](#)
- [Troubleshooting](#)

Resolution order (12-factor)

Herald follows 12-factor conventions (spec V3 §3.3 + §11.4.10) with a strict, single-direction precedence. From highest to lowest:

1. **Explicit CLI flag** — e.g. `pherald migrate up --dsn=postgres://...` always wins.
2. **Shell-exported env var** — read by Herald's Go code via `os.Getenv`. Source: your `~/ .bashrc`, `~/ .zshrc`, your CI's secret store, a Kubernetes ConfigMap / Secret, the parent shell that launched pherald, etc.

3. **Project-local .env file** — fallback only. Read automatically by docker-compose for the quickstart stack; for native pherald execution you source it explicitly (see below).
4. **Compiled default** — lowest precedence. Examples: `HERALD_DB_PORT=24100`, `HERALD_REDIS_DB=0`, `HERALD_CLAUDE_BIN=c\laude`.

The load-bearing rule: shell exports ALWAYS win over .env. This means an operator can have team defaults in .env while still overriding per-developer-host via .bashrc. The override never accidentally leaks back into .env. This is exactly the contract Herald's `commons_infra.envOr()` helper (`commons_infra/boot.go`) enforces: it calls `os.Getenv` first; if that returns empty, it falls back to a compiled default. The .env is read at one layer above — either by docker-compose or by the operator's shell when they source .env.

Setting credentials in ~/.bashrc / ~/.zshrc (shell-export path)

Append export lines to your shell's startup file. The exact file depends on your default shell:

- **macOS default (zsh):** ~/.zshrc
- **Linux default (bash):** ~/.bashrc (interactive non-login) or ~/.bash_profile (login)
- **Both shells:** ~/.profile (POSIX fallback — sourced by both bash login shells and zsh)

Example block to add (substitute real values from later sections of this guide):

```
# Herald – Postgres
export HERALD_DB_PASSWORD='your-strong-postgres-password'
# (other HERALD_DB_* vars are fine at defaults for quickstart
  compose)

# Herald – Redis
export HERALD_REDIS_PASSWORD='your-strong-redis-password'

# Herald – Telegram (HRD-011)
export HERALD_TGRAM_BOT_TOKEN='1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3oPqR4S5t'
export HERALD_TGRAM_CHAT_ID='-1001234567890'

# Herald – Claude Code (HRD-012)
export HERALD_CLAUDE_PROJECT_NAME='Herald'
# HERALD_CLAUDE_BIN defaults to 'claude' (looked up via $PATH).
# HERALD_CLAUDE_SESSION_UUID is auto-bootstrapped on first run;
  set only for testing.
```

After editing, run `source ~/.zshrc` (or `source ~/.bashrc`) or open a new terminal so the changes take effect.

Verify:

```
echo "${HERALD_TGRAM_BOT_TOKEN:0:10}..." # prints first 10 chars
      then ellipsis
# Should NOT print the empty string. If it does, the export didn't
  take.
```

Use this path for credentials that follow you across all projects + checkouts (your personal Claude session UUID; your workstation-wide Telegram bot token; etc.).

Setting credentials in .env (project-local path)

For credentials specific to one Herald checkout (a per-deploy Postgres password, a per-environment Slack webhook), use .env:

```
cd /Users/milosvasic/Projects/Herald
cp quickstart/.env.example .env
chmod 600 .env # readable only by you –
               discourages accidental cat
$EDITOR .env # fill in real values
```

The .env file is git-ignored (verified via .gitignore line 28: .env). Never git add .env. Never paste real credentials into a PR description, commit message, Slack channel, or any artifact that gets persisted.

To consume .env from **native pherald** (i.e. NOT through docker-compose), you must source it into your shell first:

```
set -a # auto-export every variable
      assigned hereafter
source .env
set +a # turn auto-export back off
pherald serve # now sees all .env vars
```

Or use direnv if you have it: echo 'dotenv' > .envrc; direnv allow. With direnv, every cd into the repo automatically exports the .env vars; every cd out unsets them.

For **compose-based** runs (docker-compose -f quickstart/docker-compose.quickstart.yml up): docker-compose (and podman-compose) reads .env automatically from the project root. No sourcing required.

Use this path for credentials specific to one Herald checkout or one deploy.

Constitutional rules — what NEVER goes in git

Per Helix Universal Constitution §11.4.10 + Herald §107:

- **NEVER** commit a .env file (anywhere — repo root, subdirectories, anywhere). Verify with `git ls-files | grep -E '^\.env$'` — must be empty.
- **NEVER** commit real bot tokens, API keys, passwords, OAuth secrets, session UUIDs.

- **NEVER** put credentials in commit messages, PR descriptions, docs/Issues.md, docs/Fixed.md, CHANGELOG, README, scripts, or any tracked artifact.
- **DO** commit quickstart/.env.example with placeholder values (change-me-..., re_XXXXXXXXXXXXXXXXXXXXXXXXXXXX, etc.) — the file’s purpose is documentation, not secret storage.
- **DO** commit code that READS env vars via os.Getenv — the code is harmless without the vars set.

A leaked credential in tracked history is a §107 release-blocker:

- If you discover one in your local working tree, do NOT commit; remove it first.
- If you discover one already pushed to a remote, **rotate the credential immediately** (regenerate the token, change the password, etc.) — git history scrubbing is forensic-quality but does not retroactively invalidate the leaked secret. Inform the operator.

Per-service credential setup

Postgres (LIVE)

REQUIRED — Herald cannot run without these. Closed by HRD-010.

Env var	Description	Default	Where to obtain
HERALD_DB_HOST	Postgres host	127.0.0.1	quickstart compose / your own deploy
HERALD_DB_PORT	Postgres port	24100	matches quickstart compose mapping 24100:5432
HERALD_DB_USER	DB role for Herald migrations	herald	created by quickstart compose’s POSTGRES_USER
HERALD_DB_PASSWORD	DB password for that role	(no default)	YOU choose — openssl rand -base64 32
HERALD_DB_NAME	Database name	herald	matches POSTGRES_DB
HERALD_PG_DSN	URL form used by pherald migrate CLI	(composed from above)	postgres://herald:<pass>@host:port/herald

Setup steps:

1. Choose a strong password: openssl rand -base64 32 (copy output).
2. Add to .env:

```
HERALD_DB_PASSWORD=<paste-the-output>
```

3. Or export from shell:

```
export HERALD_DB_PASSWORD='<paste-the-output>'
```

4. Boot Postgres via compose: `docker-compose -f quickstart/docker-compose.quickstart.yml up -d postgres`.

5. Verify: `pherald migrate status` → should print `schema version: 0` on a fresh DB.

6. Apply migrations: `pherald migrate up` → should print `applied 11 migration(s)`.

Redis (LIVE)

REQUIRED. Closed by HRD-010 (Redis health-check live; full feature wiring in subsequent HRDs).

Env var	Description	Default	Notes
HERALD_REDIS_ADDR	host:port	127.0.0.1:24200	matches quickstart compose 24200:6379
HERALD_REDIS_PASSWORD	ACL “default” user password	(no default)	YOU choose
HERALD_REDIS_DB	DB index	0	default OK for single-tenant

Setup mirrors Postgres: strong random password to `.env` or shell. Verify with the integration test `TestUp_PopulatesRedis_TTLRoundTrip` (set the env var first; run `go test ./commons_infra/ -tags=integration -run TestUp_PopulatesRedis_TTLRoundTrip -count=1`).

Telegram — HRD-011 (LIVE)

REQUIRED for the Telegram channel. Closed by HRD-011 (code complete; live evidence requires operator credentials).

Env var	Description	Default	How to obtain
HERALD_TGRAM_BOT_TOKEN	Bot API token	(no default)	@BotFather (see steps below)
HERALD_TGRAM_CHAT_ID	Target chat ID (numeric)	(no default)	/getUpdates (see steps below)
HERALD_TGRAM_LIVE_INBOUND		(unset)	set to 1 when running E19

Env var	Description	Default	How to obtain
	Enables the E19 vertical-slice test		+ hand-sending a message
HERALD_TGRAM_WEBHOOK_SECRET	Webhook secret_token (PRODUCTION)	(unset)	openssl rand -hex 32 — reserved for HRD-NNN webhook-ingress

Step 1: Create a bot via @BotFather

1. Open Telegram → search @BotFather → start chat.
2. Send /newbot.
3. Choose a display name (e.g. “Herald Operator Bot”).
4. Choose a username ending in bot (e.g. herald_operator_bot).
5. BotFather returns a token of the form
1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3oPqR4S5t.
6. **Copy the token immediately** — set HERALD_TGRAM_BOT_TOKEN=<the-token>. (BotFather will let you regenerate if you lose it; never paste the token publicly.)

Step 2: Get your chat ID

1. Add the bot to a chat (or DM it directly), then send any message to it.
2. Visit <https://api.telegram.org/bot<your-token>/getUpdates> in a browser (substitute the real token).
3. Look in the JSON response for "chat":{"id": NUMBER, ...}. That NUMBER is your chat ID.
 - For groups, the ID is negative (e.g. -1001234567890).
 - For DMs, the ID is positive and equals the user ID.
4. Set HERALD_TGRAM_CHAT_ID=<the-number>.

Step 3: (Optional) Enable live-inbound for E19 vertical-slice test

Set HERALD_TGRAM_LIVE_INBOUND=1 ONLY when you intend to hand-send a Telegram message during a test run so the Subscribe loop actually pulls an inbound update. Leave unset for everyday operation — the long-poll goroutine still runs but the E19 test SKIPS without this flag.

Step 4: (Future — webhook mode) generate a webhook secret

For production, prefer webhook over long-poll for lower latency:

```
openssl rand -hex 32      # generate a 256-bit secret
```

Set HERALD_TGRAM_WEBHOOK_SECRET=<the-hex>. Configure the bot’s webhook URL to <https://your-herald.example.com/v1/webhooks/tgram> (when HRD-NNN ships webhook ingress; long-poll only in v1).

Step 5: Verify

```
go test ./commons_messaging/channels/tgram/ -tags=integration
      -run TestHealthCheck_LiveBotAPI -count=1 -timeout=60s
```

Expected: PASS — proves the token is valid + the bot is enabled.

Then run the E17 evidence test:

```
go test ./commons_messaging/channels/tgram/ -tags=integration
      -run TestSend_PersistsDeliveryEvidence -count=1
      -timeout=300s
```

Expected: PASS — a real Telegram message arrives in your configured chat + a row appears in `outbound_delivery_evidence`.

Claude Code dispatcher — HRD-012 (LIVE)

REQUIRED for the Claude Code LLM dispatch surface. Closed by HRD-012 (live PASS evidence captured: Tasks 6 + 7 produced 24s + 36s live runs against real `claude --resume`).

Env var	Description	Default	How to obtain
HERALD_CLAUDE_BIN	Path to <code>claude</code> CLI	<code>claude</code> (PATH lookup)	install Claude Code per Anthropic instructions
HERALD_CLAUDE_PROJECT_NAME	Identifier for the consuming project	(no default)	YOU choose — typically matches your repo folder name (e.g. Herald, ATMOSphere)
HERALD_CLAUDE_SESSION_UUID	(Optional) Pre-existing Claude session UUID	(auto-bootstrapped)	bootstrap path resolves this; set explicitly only for diagnostics
HERALD_CLAUDE_WORKDIR	(Optional) Working dir for <code>cmd.Dir</code> of <code>claude --resume</code>	.	set if your consuming project is not the cwd

Step 1: Install Claude Code

Follow Anthropic’s official Claude Code install guide. Verify with:

```
claude --version
```


Should print a version like `claude 1.x.y`. If `claude` isn't on `$PATH`, set `HERALD_CLAUDE_BIN=/absolute/path/to/claude`.

Step 2: Choose a project name

`HERALD_CLAUDE_PROJECT_NAME` is used as the anchor file name (e.g. `<workdir>/.herald/claude-code/sessions/Herald.session`). MUST be a valid filename — alphanumerics + dashes + underscores. Typically you choose your repo folder name.

```
export HERALD_CLAUDE_PROJECT_NAME='Herald'
```

Step 3: Bootstrap a session (two paths)

Path A — auto-bootstrap (the §33.2-canonical path):

Just omit `HERALD_CLAUDE_SESSION_UUID`. Herald's `ResolveSession()` will spawn `claude --print "Initializing Herald session for project: <name>"`, capture the new session UUID from stdout, persist it to `<workdir>/.herald/claude-code/sessions/<project>.session`, and reuse it across runs.

Path B — pre-create a session manually (for testing / diagnostics):

```
claude --print "init"
# Watch the session ID printed at the end of stdout – copy it.
export HERALD_CLAUDE_SESSION_UUID='3e67dcd3-c66f-4687-
    b936-562191437310' # example
```

Step 4: Verify

```
go test ./commons_messaging/dispatch/claude_code/
    -tags=integration -run TestDispatch_LiveClaudeInvocation
    -count=1 -timeout=300s
```

Expected: PASS in ~20-30s — Claude receives an envelope, replies with the `<<<HERALD-REPLY>>>` JSON line, parsed with non-empty Outcome + Summary.

Slack — planned, V2

Status: NOT YET IMPLEMENTED. Spec V3 §11.2 + §11.9. When the Slack channel lands as an HRD, env vars will follow this shape:

Env var	Description
HERALD_SLACK_BOT_TOKEN	<code>xoxb-...</code> from Slack app's "Install to Workspace" flow
HERALD_SLACK_SIGNING_SECRET	For verifying inbound webhooks (Events API)
HERALD_SLACK_APP_TOKEN	<code>xapp-...</code> for Socket Mode (alternative to Events API)
HERALD_SLACK_DEFAULT_CHANNEL	

Env var	Description
	Channel ID (e.g. C012345ABCD) — Slack channels DON'T use names internally

Reserve env var names now in your `.env` (commented out) to keep `.env.example` stable when Slack lands.

Email (SMTP) — planned, V2

Status: NOT YET IMPLEMENTED. Spec V3 §11.4. When SMTP channel lands:

Env var	Description
HERALD_SMTP_HOST	mail relay hostname
HERALD_SMTP_PORT	typically 587 (STARTTLS), 465 (TLS), or 25 (plain — discouraged)
HERALD_SMTP_USERNAME	mailbox username
HERALD_SMTP_PASSWORD	mailbox password OR app-password (per provider)
HERALD_SMTP_FROM	sender address (must be authorized for the username)

Email (Resend) — planned, V2

Status: NOT YET IMPLEMENTED. Alternative to raw SMTP. Spec V3 §11.4.

Env var	Description
HERALD_RESEND_API_KEY	re_XXXXX... from Resend dashboard (Settings → API Keys)
HERALD_RESEND_FROM	sender address verified in Resend

Max — planned, V2

Status: NOT YET IMPLEMENTED. Spec V3 §11.3. Russian-market messenger.

Env var	Description
HERALD_MAX_BOT_TOKEN	TBD — Max Bot API token format
HERALD_MAX_CHAT_ID	TBD

Detailed setup will follow when Max lands as an HRD.

Microsoft Teams — planned, V3

Status: NOT YET IMPLEMENTED. Spec marks Teams as a later iteration (post-V2).

Lark / Discord / WhatsApp / Viber — planned, later iterations

Per spec V3 §11, these are reserved for later iterations. Env var names will be defined when each HRD opens. Reserve a comment block in your `.env` for “later-iteration channels” to keep your file aligned with the spec.

Alternate LLM dispatchers (OpenCode / Aider / Gemini / Cursor / Managed Agents) — planned

Status: NOT YET IMPLEMENTED. Spec V3 §33 declares Dispatcher as a pluggable interface; only `claude-code` ships in V1. Future env vars (TBD HRDs):

Env var	Dispatcher
HERALD_OPENCODE_*	OpenCode
HERALD_AIDER_*	Aider
HERALD_GEMINI_API_KEY	Gemini Managed Agent
HERALD_CURSOR_*	Cursor
ANTHROPIC_API_KEY	Anthropic Managed Agent (Bedrock / Vertex variants TBD)

Quickstart compose vs native pherald

Two ways to run Herald:

Mode	Env source	Use case
Quickstart compose	<code>docker-compose -f quickstart/docker-compose.quickstart.yml</code> up reads <code>.env</code> automatically from the repo root + interpolates shell-exported vars on top	development, integration tests, demos
Native pherald	<code>pherald serve, pherald migrate — os.Getenv</code> only; reads ONLY shell-exported vars. Source <code>.env</code> first with <code>set -a;</code> <code>source .env; set +a</code> (or use <code>direnv</code>)	production-like, performance testing, CI

In compose mode, `.env` is the canonical source. In native mode, shell exports are. Both should reach the same end state per the resolution order at the top of this doc.

Audit checklist (run before every commit)

Before `git commit`, run:

```

cd /Users/milosvasic/Projects/Herald

# 1. .env is not tracked
git ls-files | grep -E '^\.env$' \
  && echo "BLOCK: .env is tracked - REMOVE BEFORE COMMIT" \
  || echo "OK: .env not tracked"

# 2. No obvious-format secrets in tracked files
git ls-files | xargs grep -lE "ghp_[a-zA-Z0-9]{36}|sk-[a-zA-Z0-9]{48}|xox[bps]-[AKIA[A-Z0-9]{16}|6[0-9]{9}:AAH[a-zA-Z0-9_-]{30,}" 2>/dev/null \
  | grep -v '\.env\.example' \
  && echo "BLOCK: probable secret in tracked files" \
  || echo "OK: no obvious-format secrets"

# 3. .env.example has only placeholder values (no real-looking secrets)
grep -E '=[a-zA-Z0-9+/=]{40,}' quickstart/.env.example \
  | grep -vE 'change-me|xxxxx|example|<.+>|FILLME' \
  && echo "WARN: .env.example may contain real values - verify each line" \
  || echo "OK: .env.example has only placeholder values"

```

Any “BLOCK” output means do NOT commit. Rotate the leaked credential, scrub from working tree, re-run the audit, and only then commit.

Troubleshooting

“connection refused” on Postgres: The compose stack isn’t up. Run `docker-compose -f quickstart/docker-compose.quickstart.yml up -d postgres` and wait ~3s.

“SQLSTATE 28P01” (password authentication failed): The container’s stored password hash doesn’t match the env var. Either reset the volume (`podman-compose down -v` — destroys data) or run `ALTER USER herald PASSWORD '<your-env-var-value>';` inside the container.

pherald migrate up says “HERALD_PG_DSN environment variable must be set”: Compose-style env vars (`HERALD_DB_HOST/PORT/USER/PASSWORD/NAME`) are NOT automatically composed into `HERALD_PG_DSN`. Set the DSN explicitly:

```

export HERALD_PG_DSN="postgres://herald:${HERALD_DB_PASSWORD}@127.0.0.1:24100/herald"

```

Telegram integration test SKIPS even though token is set: Confirm both `HERALD_TGRAM_BOT_TOKEN` AND `HERALD_TGRAM_CHAT_ID` are set. The SKIP guard requires both.

claude is on PATH but Dispatch test still SKIPS: Confirm `HERALD_CLAUDE_PROJECT_NAME` AND (for the persistence test) `HERALD_CLAUDE_SESSION_UUID` are set. The SKIP guard layers all three checks.

“.env values not picked up by native pherald”: You must source `.env` first — pherald reads `os.Getenv` only, not `.env` files. Use `set -a; source .env; set +a` or `direnv`.

“My credential leaked into git history”: Rotate the credential IMMEDIATELY (regenerate the token, change the password). Then escalate per Universal Constitution §11.4.10 — git-history scrubbing is forensic-quality only; the leaked value is forever compromised once pushed to a remote.